13 ERRORS · WITH CODE

Error Handling in Python

A complete visual reference — from **syntax slips** to **runtime explosions**, with the exact fix.

THE JAYTECH FIELD GUIDE

THE INDEX

ERROR HANDLING in Python

13 errors · every cause, code, and fix

01 SyntaxError

02 TypeError

03 NameError

04 ValueError

05 IndexError

06 KeyError

07 ZeroDivisionError

08 AttributeError

09 ImportError

10 FileNotFoundError

11 IndentationError

12 RuntimeError

13 LogicalError

FOUNDATIONS

What is Error Handling in Python?

Error handling in Python is a way to manage errors that occur while a program is running so that the program does not crash and can continue executing smoothly using try, except, else, and finally blocks.

Why Error Handling Matters

- Prevents your program from crashing
- Handles unexpected user input safely
- Makes your code more reliable
- Improves user experience
- Helps you debug and maintain code easily
- Required for real-world applications

01

ERROR Nº 01

Syntax Error

A Syntax Error happens when Python doesn't understand your code because it breaks the rules of the Python language.

⚠ COMMON CAUSES

- Missing : after if, for, while, def, etc.
- Forgetting parentheses () in print() or function calls
- Missing or mismatched quotes (' ")
- Incorrect indentation
- Incomplete or wrongly written code

<> EXAMPLE

`SyntaxError: expected ':'`

```python
if x > 5
    print("Hello")
```

HOW TO FIX

- Read the error message carefully
- Check for missing colons, brackets, and quotes
- Ensure proper indentation
- Make sure code is complete and correctly written

02

ERROR Nº 02

Type Error

A TypeError occurs when Python tries to perform an operation on incompatible data types — indicating that the operation is not supported.

⚠ COMMON CAUSES

- Adding or combining different data types (e.g. int + str)
- Passing the wrong type of argument to a function
- Using an object in an unsupported way
- Calling a function with the wrong number of arguments

<> EXAMPLE

`TypeError: unsupported operand type(s) for +`

PYTHON

```python
x = 10
y = "5"
print(x + y)
```

HOW TO FIX

- Convert data types properly (int(), str(), float())
- Check function arguments and their types
- Use type() to debug variable types
- Read the error message carefully

03

ERROR Nº 03

Name Error

A NameError occurs when Python cannot find a variable or function name because it has not been defined or is spelled incorrectly.

⚠ COMMON CAUSES

- Using a variable before defining it
- Misspelling variable or function names
- Using the wrong case (Python is case-sensitive)
- Accessing variables outside their scope

<> EXAMPLE

`NameError: name 'x' is not defined`

PYTHON

```python
a = 15
print(x)
```

HOW TO FIX

- Define the variable before using it
- Check spelling and letter case
- Ensure the variable is accessible in the current scope
- Run previous cells/code if using notebooks

04

ERROR Nº 04

Value Error

A ValueError occurs when Python receives a correct data type but the value itself is invalid or inappropriate.

⚠ COMMON CAUSES

- Converting invalid values (e.g. int("abc"))
- Passing an invalid value to a function
- Incorrect input format
- Values outside the expected range

<> EXAMPLE

ValueError: invalid literal for int()

PYTHON

```python
x = int("abc")
print(x)
```

HOW TO FIX

- Ensure the value matches the expected format
- Validate user input before conversion
- Use try-except to handle invalid values
- Print and check values while debugging

05

ERROR Nº 05

Index Error

An IndexError occurs when you try to access an index that does not exist in a list, tuple, or string. This happens when the index is out of range.

⚠ COMMON CAUSES

- Accessing an index outside the range
- Using a negative index incorrectly
- Looping beyond the length of a list
- Assuming an element exists when it doesn't

<> EXAMPLE

`IndexError: list index out of range`

PYTHON

```python
nums = [1, 2, 3]
print(nums[5])
```

HOW TO FIX

- Check the length using len()
- Make sure the index exists before accessing
- Use proper loop conditions
- Handle cases with try-except if needed

06

ERROR Nº 06

Key Error

A KeyError occurs when you try to access a dictionary key that does not exist. It typically happens when you reference a missing key.

⚠ COMMON CAUSES

- Accessing a missing key in a dictionary
- Typo or wrong key name
- Using the wrong letter case (keys are case-sensitive)
- Assuming a key exists without checking

<> EXAMPLE

`KeyError: 'email'`

PYTHON

```python
data = {"name": "Akash", "age": 20}
print(data["email"])
```

HOW TO FIX

- Check if the key exists using 'in'
- Use .get() to avoid errors
- Verify key names and spelling
- Handle missing keys with try-except

07

ERROR Nº 07

Zero Division Error

A ZeroDivisionError occurs when you attempt to divide a number by zero, which is not permitted in Python.

⚠ COMMON CAUSES

- Dividing by 0
- Using a variable whose value becomes 0
- Incorrect calculations
- Incorrect user input

<> EXAMPLE

`ZeroDivisionError: division by zero`

PYTHON

```python
x = 10
y = 0
print(x / y)
```

HOW TO FIX

- Check if the value is zero before dividing
- Validate user input
- Use try-except to handle division safely

08

ERROR Nº 08

Attribute Error

An AttributeError occurs when you attempt to access an attribute or method that an object does not define.

⚠ COMMON CAUSES

- Calling a method that doesn't exist for that data type
- Misspelling method or attribute names
- Using the wrong object type
- Assuming all objects have the same methods

<> EXAMPLE

`AttributeError: 'int' object has no attribute 'append'`

PYTHON

```
x = 10
x.append(5)
```

HOW TO FIX

- Check which methods belong to the object
- Use type() to confirm the object type
- Fix spelling mistakes
- Use the correct data type for the operation

09

ERROR Nº 09

Import Error

An ImportError happens when Python cannot find or load a module, or when a requested item does not exist within that module.

⚠ COMMON CAUSES

- Module is not installed
- Wrong module or package name
- Trying to import something that doesn't exist
- Incorrect project or file structure

<> EXAMPLE

`ImportError: cannot import name 'sqr'`

PYTHON

```python
from math import sqr
result = sqr(9)
print(result)
```

HOW TO FIX

- Check module and function names carefully
- Install missing modules using pip
- Verify file and folder structure
- Use the correct import statement

10

ERROR Nº 10

File Not Found Error

A FileNotFoundError occurs when Python attempts to open or access a file that cannot be found, often due to an incorrect file name or file path.

⚠ COMMON CAUSES

- File does not exist
- Wrong file name or extension
- Incorrect file path
- File is in a different directory

<> EXAMPLE

`FileNotFoundError: No such file or directory`

PYTHON

```
file = open("data.txt", "r")
content = file.read()
file.close()
```

HOW TO FIX

- Check the file name and extension
- Verify the file path
- Make sure the file exists in the directory
- Use absolute paths if needed

11

ERROR Nº 11

Indentation Error

It occurs when Python finds incorrect spacing (indentation) in code. Python uses indentation to understand code, so proper spacing is mandatory.

⚠ COMMON CAUSES

- Missing indentation after if, for, while, def, etc.
- Mixing tabs and spaces together
- Extra or inconsistent spaces
- Code block not aligned properly

<> EXAMPLE

`IndentationError: expected an indented block`

PYTHON

```
if x > 5:
print("Hello")
```

HOW TO FIX

- Use consistent indentation (recommended: 4 spaces)
- Don't mix tabs and spaces
- Ensure all code inside a block is properly aligned
- Use a code editor that auto-formats indentation

12

ERROR Nº 12

Runtime Error

A RuntimeError occurs when an error happens while the program is running, even though the code syntax is correct.

⚠ COMMON CAUSES

- Logical mistakes in code
- Infinite loops or recursion issues
- Resource limitations (memory, time)
- Incorrect use of built-in functions

<> EXAMPLE

`RuntimeError: maximum recursion depth exceeded`

PYTHON

```python
def recurse():
    return recurse()
recurse()
```

HOW TO FIX

- Review program logic carefully
- Add proper conditions to stop loops or recursion
- Optimize code to reduce resource usage
- Use debugging and error messages to trace the issue

13

ERROR Nº 13

Logical Error

A Logical Error occurs when your code runs without errors but produces incorrect output.

⚠ COMMON CAUSES

- Wrong operator precedence
- Off-by-one mistakes in loops
- Incorrect formula or condition
- Bad assumptions about inputs

<> EXAMPLE

Output: 0 (expected: 10)

PYTHON

```python
x = 10
y = 5
print(x - y * 2)
```

HOW TO FIX

- Recheck logic and conditions
- Use print statements to debug values
- Break the code into smaller parts
- Test with multiple inputs

THAT'S A WRAP

Python Errors Mastered Visually

You now have a complete field guide for Python error handling — from syntax slips to tricky runtime issues.

Repost Save Comment

Follow JayTech AI →